

Lab Task 14

University News App — Fetching REST API with Retrofit

Android Studio • Kotlin • Retrofit2 • Coroutines • Glide



Instructions

1. Complete the task following all requirements carefully.
2. After completion, push your project to your GitHub repository.
3. UI must be clean, aligned, and properly spaced.
4. Use the free JSONPlaceholder API (jsonplaceholder.typicode.com) as your data source.
5. Handle all loading, success, and error states in the UI.



Scenario

You are building a University News & Posts App that fetches posts, users, and comments from a public REST API. The app uses Retrofit2 to make HTTP requests, Kotlin Coroutines for async operations, and Glide for loading avatar images. Students can browse all posts, view post details with comments, explore user profiles, and search for posts by title. This lab teaches the full Retrofit setup: API interface, data models, service layer, and error handling.



Project Setup

Project Name	UniversityNewsApp
Template	Empty View Activity
Language	Kotlin
API Base URL	https://jsonplaceholder.typicode.com/

Step 1 — Add Dependencies

```
// build.gradle (App level)
dependencies {
    // Retrofit + Gson converter
    implementation 'com.squareup.retrofit2:retrofit:2.9.0'
    implementation 'com.squareup.retrofit2:converter-gson:2.9.0'

    // OkHttp logging interceptor (for debugging)
    implementation 'com.squareup.okhttp3:logging-interceptor:4.12.0'

    // Coroutines
    implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3'
    implementation 'androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0'
    implementation 'androidx.lifecycle:lifecycle-runtime-ktx:2.7.0'

    // Glide (image loading)
    implementation 'com.github.bumptech.glide:glide:4.16.0'

    // RecyclerView + CardView
    implementation 'androidx.recyclerview:recyclerview:1.3.2'
    implementation 'androidx.cardview:cardview:1.0.0'
}
```

Step 2 — Add Internet Permission

```
<!-- AndroidManifest.xml -->
<uses-permission android:name="android.permission.INTERNET" />
```



API Reference

This app uses the free JSONPlaceholder API. No API key is required. The following endpoints are used:

Method	Endpoint	Description
GET	/posts	Fetch all posts (100 items)
GET	/posts/{id}	Fetch a single post by ID
GET	/posts/{id}/comments	Fetch all comments for a post
GET	/users	Fetch all users
GET	/users/{id}	Fetch a single user profile
GET	/users/{id}/posts	Fetch all posts by a user

✓ Requirements & Screens

Retrofit Setup — Required Architecture

Create the following files in a structured package layout:

```
app/src/main/java/com/university/newsapp/  
├── model/  
│   ├── Post.kt  
│   ├── User.kt  
│   └── Comment.kt  
├── network/  
│   ├── ApiService.kt           ← Retrofit interface  
│   └── RetrofitClient.kt       ← Singleton Retrofit instance  
├── repository/  
│   └── PostRepository.kt       ← Abstracts network calls  
└── ui/  
    ├── MainActivity.kt  
    ├── PostDetailActivity.kt  
    └── UserProfileActivity.kt
```

Data Models

```
// Post.kt  
data class Post(  
    val id: Int,  
    val userId: Int,  
    val title: String,  
    val body: String  
)  
  
// User.kt  
data class User(  
    val id: Int,  
    val name: String,  
    val username: String,  
    val email: String,  
    val phone: String,  
    val website: String,  
    val company: Company  
)  
data class Company(val name: String, val catchPhrase: String)  
  
// Comment.kt  
data class Comment(  
    val id: Int,  
    val postId: Int,  
    val name: String,  
    val email: String,  
    val body: String  
)
```

Screen 1 — Post List (MainActivity)

Layout: ConstraintLayout + RecyclerView

- Toolbar with app title "University News"
- SearchView to filter posts by title (filter locally after loading)
- RecyclerView with custom PostAdapter — each card shows: post title (bold), body excerpt (2 lines, ellipsized), user ID badge, post ID
- SwipeRefreshLayout to allow pull-to-refresh that re-fetches posts from API
- 3 UI states to handle:
 - Loading: show centered ProgressBar
 - Success: show RecyclerView, hide ProgressBar
 - Error: show error TextView + Retry button
- Bottom navigation or tab layout with 2 tabs: Posts and Users
- Tapping a post opens PostDetailActivity

Screen 2 — Post Detail (PostDetailActivity)

Layout: NestedScrollView + LinearLayout

- Receives post ID via Intent
- Displays full post: title (large, bold), full body text
- Author card — fetches user details using /users/{userId} and shows name, email, company
- Tapping author card opens UserProfileActivity
- Comments section — fetches /posts/{id}/comments and renders in a nested LinearLayout (or another RecyclerView)
- Each comment card shows: commenter name, email, body text
- Show separate loading indicators for the post, author, and comments sections

Screen 3 — Users List (UsersFragment or UsersActivity)

Layout: RecyclerView

- Fetches /users and displays in a RecyclerView
- Each user card: avatar circle (generate initials — e.g. 'JD' for John Doe), name, username, email
- Tapping opens UserProfileActivity

Screen 4 — User Profile (UserProfileActivity)

Layout: ConstraintLayout + NestedScrollView

- Avatar circle at top with user initials (large, colorful background)
- Name, username (@handle), email, phone, website, company name and catchphrase
- "Posts by this user" section — fetches /users/{id}/posts and shows a mini list
- Tapping a post in the mini list opens PostDetailActivity



Code Hints

RetrofitClient.kt — Singleton Setup

```
object RetrofitClient {  
    private const val BASE_URL = "https://jsonplaceholder.typicode.com/"  
  
    private val loggingInterceptor = HttpLoggingInterceptor().apply {  
        level = HttpLoggingInterceptor.Level.BODY  
    }  
  
    private val okHttpClient = OkHttpClient.Builder()  
        .addInterceptor(loggingInterceptor)  
        .connectTimeout(30, TimeUnit.SECONDS)  
        .readTimeout(30, TimeUnit.SECONDS)  
        .build()  
  
    val instance: ApiService by lazy {  
        Retrofit.Builder()  
            .baseUrl(BASE_URL)  
            .client(okHttpClient)  
            .addConverterFactory(GsonConverterFactory.create())  
            .build()  
            .create(ApiService::class.java)  
    }  
}
```

ApiService.kt — Retrofit Interface

```
interface ApiService {  
    @GET("posts")  
    suspend fun getAllPosts(): List<Post>  
  
    @GET("posts/{id}")  
    suspend fun getPostById(@Path("id") id: Int): Post  
  
    @GET("posts/{id}/comments")  
    suspend fun getCommentsByPost(@Path("id") postId: Int): List<Comment>  
  
    @GET("users")  
    suspend fun getAllUsers(): List<User>  
  
    @GET("users/{id}")  
    suspend fun getUserById(@Path("id") id: Int): User  
  
    @GET("users/{id}/posts")  
    suspend fun getPostsByUser(@Path("id") userId: Int): List<Post>  
}
```

Calling the API with Coroutines

```
// In Activity or ViewModel using lifecycleScope
private fun loadPosts() {
    progressBar.visibility = View.VISIBLE
    recyclerView.visibility = View.GONE
    errorText.visibility = View.GONE

    lifecycleScope.launch {
        try {
            val posts = RetrofitClient.instance.getAllPosts()
            progressBar.visibility = View.GONE
            recyclerView.visibility = View.VISIBLE
            adapter.submitList(posts)
        } catch (e: HttpException) {
            showError("Server error: ${e.code()}")
        } catch (e: IOException) {
            showError("Network error. Check your connection.")
        } catch (e: Exception) {
            showError("Something went wrong: ${e.message}")
        }
    }
}

private fun showError(message: String) {
    progressBar.visibility = View.GONE
    errorText.visibility = View.VISIBLE
    errorText.text = message
    retryButton.visibility = View.VISIBLE
}
```